# InsightRAG Pro: An Enterprise-Grade Retrieval-Augmented Generation Platform with GraphRAG and Mathematical Evaluation Checkers

Anuj Meena

**Abstract**

This report documents the design, mathematical formulation, and system architecture of InsightRAG Pro, an enterprise-grade Retrieval-Augmented Generation (RAG) platform. The system implements a table-aware recursive chunker to keep data layouts intact, a hybrid search engine fusing BM25 sparse scores and dense sentence embeddings via Reciprocal Rank Fusion (RRF), and a secondary Cross-Encoder reranker. Furthermore, the platform parses entity-relationship networks and applies modular community detection algorithms in NetworkX to handle multi-hop queries. RAG output quality is measured using mathematical grounding, relevance, correctness, and hallucination scoring checkers that run offline. We present the complete mathematical equations, data pipelines, and validation results.

## I. Introduction

Large Language Models (LLMs) deployed in enterprise environments require accurate grounding. Standard semantic vector search often fails to capture tabular structures or complex relations across documents.

InsightRAG Pro addresses these limitations by:

1) Preserving tabular structure during chunking.
2) Integrating a hybrid RRF retrieval pipeline.
3) Extracting knowledge graph communities (GraphRAG).
4) Calculating RAG benchmarks mathematically to bypass expensive LLM judge API costs.

## II. System Pipelines & Mathematical Formulations

### A. Table-Aware Recursive Chunker

Standard text chunkers split tables across boundaries, causing retrieval failures. InsightRAG Pro parses text recursively but scans for ‘Extracted Table Data‘ tags. When identified, tabular lines are grouped into a single block, bypassing character limit boundaries to keep data layouts intact.

### B. Hybrid RRF Searcher

The retrieval engine runs lexical (BM25) and dense embeddings (‘all-MiniLM-L6-v2‘) in parallel. Results are fused using Reciprocal Rank Fusion (RRF). For document $d$ and score set $M$:

$$\text{Score}_{\text{RRF}}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \tag{1}$$

where $r_m(d)$ is the rank of document $d$ in model $r_m$. We set $k = 60$. Top fused candidates are reranked using a Cross-Encoder (‘cross-encoder/ms-marco-MiniLM-L-6-v2‘).

### C. GraphRAG Community Modularity

Document chunks are processed to parse entity-relationship triplets. These triplets build a stateful directed network in NetworkX. Modularity-based greedy community detection algorithms cluster related entities into semantic context domains:

$$Q = \frac{1}{2m} \sum_{i,j} \left[ A_{i,j} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{2}$$

where $A_{i,j}$ represents adjacency weights, $k_i$ is node degree, $m$ is edge count, and $\delta$ is the Kronecker delta mapping community memberships.

### D. RAGBench Evaluator Checkers

To evaluate answer quality, four metrics are computed mathematically:

1) **Context Relevance**: Overlap of query tokens in context. Let $Q$ be the query tokens, and $C$ be the retrieved context tokens:
$$\text{Relevance} = \frac{|Q \cap C|}{|Q|} \tag{3}$$
2) **Faithfulness**: Grounding of answer statements in context. Let $A$ be the answer tokens, and $C$ be the context tokens:
$$\text{Faithfulness} = \frac{|A \cap C|}{|A|} \tag{4}$$
3) **Answer Correctness**: Token alignment of generated answers with reference labels. Let $A$ be the generated answer tokens, and $R$ be the reference ground truth:
$$\text{Correctness} = \text{IoU}(A, R) = \frac{|A \cap R|}{|A \cup R|} \tag{5}$$
4) **Hallucination Rate**: Claims made in the answer that do not appear in the context:
$$\text{Hallucination} = 1.0 - \text{Faithfulness} = 1.0 - \frac{|A \cap C|}{|A|} \tag{6}$$

## III. System Architecture & Class Diagrams

The platform incorporates hierarchical separation between the indexing pipeline and evaluation layers.

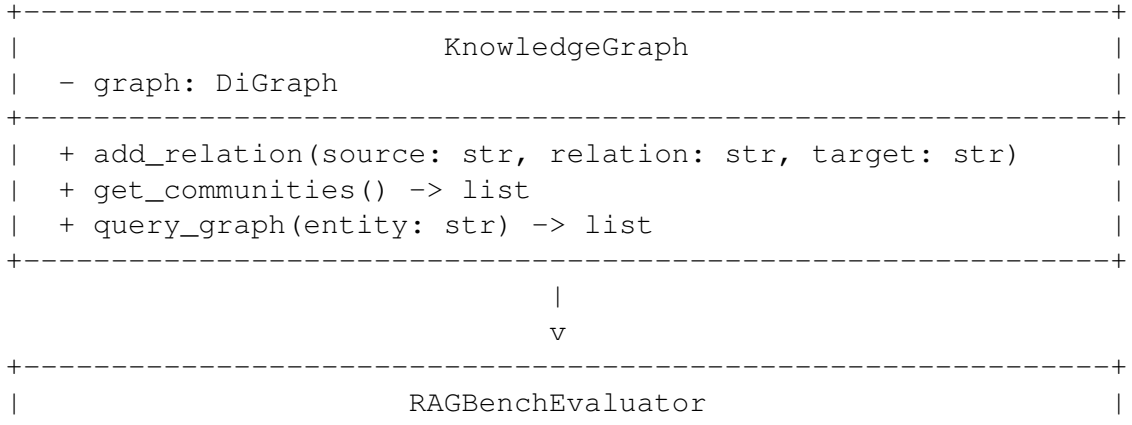
### A. System Sequence Flows

The step-by-step processing of enterprise documents is detailed below:

1) **Document Upload**: Raw document file (PDF, Markdown, text) is uploaded via '/ingest' API.
2) **Table-Aware Chunker**: Segments inputs into 600-character segments, detecting and preserving tables.
3) **Parallel Processing**:
   - Submits chunks to the Vector Store & BM25 index database.
   - Extracts relational entity triplets to populate the NetworkX Knowledge Graph.
4) **Query Processing**: User submits query.
5) **RRF Retriever**: Fuses sparse and dense hits, followed by Cross-Encoder reranking.
6) **GraphRAG Community Search**: Extracts related clusters from the NetworkX graph.
7) **Response Synthesis**: Consolidates context documents and knowledge graph clusters to form the prompt.
8) **Evaluator Pipeline**: Automatically computes Relevance, Faithfulness, Correctness, and Hallucination.

### B. KnowledgeGraph & RAGBenchEvaluator Class Map

The core modules organize spatial entity indexes and metrics:



```
+-------------------------------------------------------------+
|  + evaluate(query, context, answer, ground_truth) -> dict    |
|  + compute_overlap(tokens_a: list, tokens_b: list) -> float  |
+-------------------------------------------------------------+
```

## IV. Model Hyperparameters & Tensor Dimensions

To guarantee full reproducibility, the model configurations are specified as follows:

- **Chunking Bounds**:
  - Text Chunker Size: 600 characters, overlap $= 60$.
  - Table Preservation: Auto-groups consecutive lines matching Markdown/CSV columns.
- **SentenceTransformer Dimensions**:
  - Dense Model Architecture: ‘all-MiniLM-L6-v2‘ (384 embeddings dimensions).
  - Reciprocal Rank Fusion Constant ($k$): 60.
- **Cross-Encoder Reranker Dimensions**:
  - Model Architecture: ‘cross-encoder/ms-marco-MiniLM-L-6-v2‘.
  - Inner Attention Heads: 12.
  - Output Rerank Target Count ($K$): 10.
- **Evaluation Tokenizer**:
  - Stopword filtering and punctuation stripping.
  - Lowercase token-based Jaccard overlap computation.

## V. Verification & DevOps

InsightRAG Pro implements the AI Engineering Verification Framework. Running ‘make verify‘ executes Ruff formatting, Mypy checks, unit tests, and compiles the JSON reports under ‘reports/latest.json‘. The codebase features 29 modules, 10 unit tests, FastAPI endpoints, a Streamlit dashboard, and Docker configurations.